NOMILA-V2 · AUDIT · 10 SEP 2026

M.I.A. Score Engine Audit

How an athlete comes into existence, what feeds the marketability score and the rate card, how they are computed, and every process that reads them. Written so the scoring methodology can be replaced with the least possible disturbance downstream.

branch main @ 49b236d engine scoringService v7 · weights v7.2.0

rate model v3.1.0-calibrated 10 scored athletes in prod

THE CORE FINDING

The score is well-contained on the way in, loosely coupled on the way out

One pure function computes it, one writer persists it. But 40+ readers consume the stored columns by shape, and most of them fail silently rather than loudly if the shape changes.

THE CONTRACT TO PROTECT

displayScore 200–850 · five tier words · four capitals /100

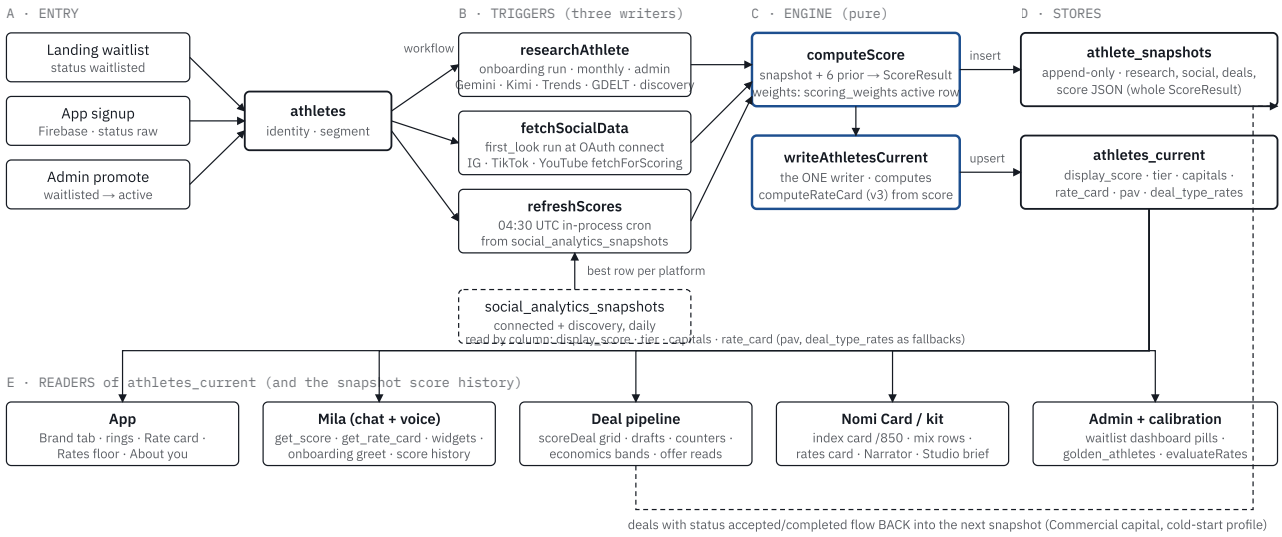
Those three things are read by the app, Mila, the voice worker, the deal scorer, the Nomi Card and the rate model. Keep them and a new algorithm is a weights-row swap. Change them and it is a cross-cutting migration.

THE HIDDEN COUPLING

Rates are a function of the score

The v3 rate card multiplies market benchmarks by a modifier built from the capitals and the tier word. A new score changes every rate, every deal verdict and every draft counter unless the rate side is re-calibrated with it.

The pipeline in one picture



Three entry doors make an athletes row. Three writers each build a scoring snapshot, call the same pure engine, insert an append-only snapshot, and upsert the current row through one writer that also derives the rate card. Everything downstream reads the stored columns, and signed deals feed the next score.

A · How an athlete comes into the system

There is no "create user" service. An athlete is an `athletes` row plus an `account_roles` row, and three doors create or claim one. Auth is Firebase everywhere (Google, Apple, Microsoft code exchange); the server never stores a password.

DOOR	ROUTE	WHAT IT WRITES	STATUS AFTER	WHAT STAR
Landing page waitlist	`POST /api/v1/waitlist`	Upserts by Firebase uid, one partial payload per step: name, school, sport, goals, career status, typed social handles, referral code	`waitlisted`	An **onboard** workflow ru moment a s is known (idempoter `onboardir <id>`)
App signup	`POST /api/v1/auth/signup`	Returns the existing row for the uid; else claims a row by e-mail; else claims an unclaimed pre-	`raw or claimed`	Onboarding only for a b new row wi sport (the a sends nam today, so us nothing)

		seeded profile by name (+sport) in status raw/enriched; else inserts a new row. Sets Firebase custom claims.		
Admin promote	`POST /api/v1/admin/waitlist/:id/promote`	Flips status to active	`active`	Onboarding none exists **research_r** run when re research is requested

What the onboarding run does (workflow v2)

The Postgres workflow runner ticks every minute from Cloud Scheduler. The onboarding run is: **research** (skipped if a research snapshot is under 7 days old) → **context_ready** (drops the cached generic Nomi Card) → **greet** (parks until the athlete reacts in chat, voice or form) → **doors** → **wrap**. The research step is the first place a score is computed. It also flips status raw → enriched and fills any empty sport, school, conference or division from what research found.

What connecting a platform does

The OAuth callback starts a **first_look** run per connection: **fetch_social** (the connect-time fetch, which computes a second score with the platform's real numbers), **backfill_recent** (30 posts) and **compute_first_look** (what the posts say, not a scoring input). Gmail and Outlook never score; Spotify keeps an identity-only fetch.

Research and connect are the only two event-driven scorings. After that the athlete is re-scored nightly and re-researched monthly, and nothing else ever touches the score.

B · What feeds the score and rates

Every input lands on an `athlete_snapshots` row before it is scored. The engine never queries the database; it receives one snapshot, up to six prior snapshots, and the athlete's level record.

INPUT	SOURCE / API	WHERE IT IS STORED	REFR

Followers, engagement rate, posts in last 30 days, verified flag, per platform	Instagram Graph (Instagram Login), TikTok, YouTube Data + Analytics APIs via the athlete's OAuth; public Business Discovery + YouTube Data API for unconnected handles	`social_analytics_snapshots` (one row per athlete × platform × day, source connected / discovery / research) → copied into `snapshot.social` with a `_computed` block	Daily UTC conn
Achievements, press mentions, brand mentions, sentiment, media assets, story arc, teams	Gemini with Google Search grounding (4 queries) → Kimi extraction (4 parallel) → union with the last 5 research snapshots	`snapshot.research`	At onbo mon the 1 05:0 adm rese
Google Trends interest score 0–100 and related queries	google-trends-api (unofficial scraper)	`snapshot.google_trends_score`	With rese
GDELT article count and average tone	GDELT free API	`snapshot.gdelt_article_count` ; tone only in `snapshot.sources.enrichment.gdelt_avg_tone`	With rese
Signed deals (status accepted or completed) with total guaranteed value and settled flag	`deals` table, written by the inbox summarizer, contract upload or self-report	`snapshot.deals` (re-read on every write)	Ever scor
Segment, division, conference	`athlete_affiliations` via `levelRecordFor` (teams, not the school column)	Passed as the level record	Ever scor
Snapshot history (up to 6 prior rows)	`athlete_snapshots` ordered by created_at	—	Ever scor

Weights	`scoring_weights` active row (v7.2.0), cached in process at startup	—	On c or re

> **Not inputs, despite appearances.** The analytics display layer (reach, saves, demographics) never reaches the engine. First look is stored on the snapshot but is not scored. Athlete-asserted facts and affiliations reach the engine only through the level record. `brand_voice` is never written.

C · How the score is computed today (v7.2)

Four capitals, each 0–100, blended by weights that depend on which data exists, then multiplied by a brand-safety factor, then stretched onto a 200–850 display scale with five tier words. The whole engine is `server/services/scoringService.ts`, pure and synchronous.

```
internal = round( Σ capital × weight[profile] ) × safety → display = piecewise(inter
```

CAPITAL	BUILT FROM	NOTES
Audience	log10(total followers)×13 capped at 100, plus tier-relative engagement, posting frequency, platform count, engagement-momentum bonus, and five history signals (consistency, stability, growth, audience quality, cross-platform)	Zero when there are no followers. Only Instagram and TikTok carry engagement; X and YouTube count toward reach only.
Athletic	Achievements classified national / conference / academic by keyword lists, plus competition level (pro 25 … youth 3), plus institutional tier (power conference 15, other D1 3), plus GDELT baseline press	The keyword lists (all-acc, all-pac, all-sec…) are conference words hard-coded in the engine, which sits against the no-hardcoding rule.
Brand	Brand mentions (confidence > .3, not team apparel), press volume + national-outlet bonus, Trends baseline, decaying Trends spike, GDELT delta and quality tiers, media assets, verified badge. Story-arc bonus is 0 since v7.1.	Spike and delta terms compare against the immediately previous snapshot, which since the nightly refresh is usually yesterday, so they are almost always 0.

Commercial	Count of earned deals, total value tiers, an engagement boost on value, paid ratio	Counted at signature. Mia's gorjana agreement is why her Commercial reads 70; every other athlete is 0.

Cold-start profiles (which weights apply)

PROFILE	CONDITION	AU / AT /
full	social + research + ≥1 earned deal	.25 / .25
socialAndResearch	social, no deals	.30 / .25
socialOnly	social, no research	.45 / .10
researchOnly	no social, no deals	.05 / .40

Only Mia is "full". The profile flips on the length of the deals array, which is why an empty array once dropped her from 634 to 576 (pinned by `emptyDealsChangesProfile.test.ts`).

Display scale and tiers

TIER	DISPLAY ≥	IN PROD TODAY
Icon	800	0
Elite	700	0
Established	550	4
Rising	400	2
Local	200	4

Safety multiplier: GDELT tone < 0 → .95, < −2 → .90, < −5 → .85; with no tone, 3+ negative sentiment items → .95.

What the engine returns and what is persisted

```
ScoreResult {
  internalScore 0–100      → athletes_current.internal_score (numeric, as a string)
  displayScore  200–850    → athletes_current.display_score
  tier          Icon|Elite|Established|Rising|Local → athletes_current.tier
  capitals { audience, commercial, brand, athletic } 0–100 → athletes_current.capitals
  ptsToNextTier, tierFillPct, safetyMultiplier, coldStartProfile, momentum{…}
}                        → the WHOLE object is stored as athlete_snapshots.score
```

`momentum`, `coldStartProfile`, `ptsToNextTier` and `tierFillPct` have no reader anywhere in the app, the kit, Mila or the voice worker. They are logged and stored, nothing more. Older snapshot rows also carry `pav`, `dealTypeRates` and `enrichmentBreakdown` from the retired v7 rate model.

D · How rates are computed, and why they depend on the score

The rate card (`rateService.ts` , v3.1.0-calibrated) prices each deal type on its own market basis. It reads nothing from the database and none of the scoring weights; its benchmarks are constants in `rateModelConfig.ts` . It is computed inside `writeAthletesCurrent` on every score write, so a rate card can never exist without a score.

```
socialPost = CPMcurve(platform, followers) × athletePremium(followers) × clamp(√(ER
```

DEAL TYPE	BASE	WHERE THE SCORE ENTERS	OUTPUT ON THE CARD
Social post (per platform)	Log-log interpolation of IMH per-post anchors × athlete premium (2.0–2.6× at micro, 1.0× at 2M)	S = narrative .40 + athletic .30 + brand .25 + commercial .05, bounds .85–1.25	`social[]` , `headlineSocial` (strongest platform)
Appearance	Flat prestige tier P1–P7	Tier **word** gates P1/P2/P4 (pro+Icon, pro+Elite, power-D1+Elite); S = athletic .45 + brand .30 + narrative .25	`appearance.ratePerEvent`
Ambassador	3.08 × headline social mid, per month	Inherits the social modifier	`ambassador.monthlyRetainer`
Autograph	Per-item tier × batch (25 NIL, 150 pro)	Tier word gates proIcon / proElite / collegeName; S = athletic .55 + narrative .25 + brand .20	`autograph.ratePerDeal`
Licensing	Royalty % (score-independent) + guaranteed minimum	S = brand .35 + audience .35 + narrative .30 on the minimum only	`licensing.guaranteedMinimum`
PAV (projected	Probability-weighted mix of the above	Through every component	`pav` ; also projected into the `pav` column and `deal_type_rates.projectedAnnualValue`

annual value)	by bucket (NIL micro / macro / pro), floors for pro and power-D1

> **Narrative is a fifth capital that only rates see.** `narrativeCapital = min(100, trends×.6 + min(40, gdelt×2))`. It is computed in `rateService` (and, unused, in the engine) and is never stored or shown.
>
> **Two silent defaults.** A capital missing from the object defaults to 50 inside `scoreMod`, and a tier word the code does not know simply fails every gate. A new score that renames a capital or a tier will change every rate with no error anywhere.

Mia today, from the live row: Instagram $497 mid, TikTok $863 mid (headline), appearance $1,005 (P5), ambassador $2,659/month, PAV $22,361 mid. Range widths come from data provenance (connected 1.0 → research 0.5) times a per-type benchmark quality, not from the score.

E · The three writers and the one writer

Three skills compute a score. All three insert an `athlete_snapshots` row and then call `writeAthletesCurrent`, which is the only code allowed to touch `athletes_current`. That discipline is what makes a transition tractable: there is one place where the score becomes columns.

WRITER	TRIGGER WORD	SOCIAL IT SCORES WITH	RESEARCH IT SCORES
`researchAthlete`	`signup · manual · re_enrich`	Connected analytics (best row per platform, 45-day fresh) with discovery filling gaps	Fresh Gemini/Kimi unio with the last 5 snapshot

`fetchSocialData`	`social_connect`	Previous snapshot's social with this platform replaced by the live OAuth fetch	Inherited from the previ snapshot
`refreshScores`	`scheduled_refresh`	Rebuilt from `social_analytics_snapshots` (connected > provider > discovery > research)	Inherited from the lates snapshot

The writer always recomputes the v3 rate card and projects `pav` and `deal_type_rates` from it, sets follower columns from the social it scored with, and stamps `score_updated_at`. Its column list is pinned by `athletesCurrentWriter.test.ts`.

Two operational facts worth knowing: the 04:30 refresh is an in-process node-cron, not a Cloud Scheduler job, and unlike the inbox sync it takes no advisory lock, so with up to three Cloud Run instances it can run up to three times (the unchanged-skip makes that harmless today). The manual endpoint `POST /api/internal/score-refresh-cron?dryRun=true` returns per-athlete before/after deltas without writing, which is the readiest tool for a shadow comparison.

F · Everything downstream, by what it reads

Grouped by the stored field, because that is what a new algorithm has to keep or migrate. "Fails how" is what happens if the field's shape or vocabulary changes without the reader being updated.

display_score and tier

READER	USES IT FOR	FAILS HOW

`GET /api/v1/athletes/:id/score` → `use-score` hook → Brand tab, Score card, Score screen, Mila widget	Headline number "of 850", tier chip, ring header	Renders whatever number arrives; "of 850" and the /100 capital labels are literals in the client
Nomi Card assembler → index card, all three themes, kit renderer	`value / max: 850`, `TIER_DISPLAY[tier]` lookup, aria text "Marketability Index X/850"	Unknown tier word falls back to the raw string; max is hard-coded 850
Narrator, shot list, Studio brief	Tier word in the model prompt ("Established tier", the deck register)	Prompt quality only
`dealDetector.scoreDeal`	"M.I.A. Score: X (tier)" line in the deal-scoring prompt	Prompt quality only
`chat.ts` (v1 inline prompt path)	Score, tier, capitals, pav in the system context	Prompt quality only
Mila `get_score` → `score_breakdown` widget; voice `/score` and the worker's score card	displayScore, tier, top capital, all four capitals	Widget and voice card are suppressed if any capital key is missing (silent)
Voice `/score-history`	Reads `athlete_snapshots.score.displayScore` and `.tier` across runs for "how has my score moved"	Any change to the stored JSON key names breaks history reads for old rows
Mila onboarding greet block, About you (`athleteRecord`), admin waitlist dashboard, rate service prestige/autograph gates	Score + tier text; tier word as a gate	Text degrades; rate gates silently drop to lower tiers

capitals { audience, commercial, brand, athletic }

READER	USES IT FOR	FAILS HOW
`ConcentricRingScore` (client), Mila widget, voice card	Four rings with locked colours (blue, green, pink, gold) and quality	Extra keys ignored; missing keys read as 0 in the client,

	labels at 80/60/40/20 on a /100 scale	suppress the card in Mila and voice
Nomi Card mix card + index rings	Grade per capital (value thresholds), Commercial row framed by confirmed-partner state	Same four keys assumed in order
`rateService.scoreMod`	Weighted blend per deal type, plus narrative	Missing key → 50 (silent)
Shot list prompt, chat context, About you Capitals line	Text	Prompt quality only

rate_card (v3 JSON) and its projections pav, deal_type_rates

READER	USES IT FOR	FAILS HOW
`dealDetector.scoreDeal` → `dealScoreGrid`	Per-type ranges (via `deriveDealTypeRates`), floor override from `deal_preferences.minimum_rates`, economics band ABOVE/UPPER/LOWER/BELOW, the verdict prose stored on `inbox_threads.score`	Old thread scores the rate figure bal into their prose u scored (268 threa today; `_rescoreImpact` exists for this)
`dealDraft`, `routes/deals` counter drafts	Unit-matched market rate for the negotiation draft	Draft quotes mov the rate
`dealValueStore.economicsReadForDeals`	Per-year deal money against PAV (Classic tab economics line)	Bands move with rate
`dealVoice.resolveAthleteRate`, Mila `offerMoney`, voice deal brief, voice profile	`headlineSocial.ratePerPost.mid` as "her rate", rate multiples spoken aloud	Numbers move; n failure
Mila `get_rate_card`, `get_score` summary	Compact projection of the card	Defensive against drift
Client Rate card block, Rates & floor settings, Brand tab PAV line	Per-platform and per-type ranges; floors set against suggested mids	Fall back to `deal_type_rate` when the card is a
Nomi Card rates card, `deckRates`, athlete context	PAV and headline per-post on the deck	Cached kits (`media_kits`, 9 and frozen shares (`card_shares`) k old figures until re

Golden athletes + `evaluateRates.ts`	Backtest of the rate model against IMH bands and published estimates (5 golden rows)	Calibration harne a runtime reader

G · Live state, 10 Sep 2026

FACT	VALUE
athletes rows by status	
athletes_current rows / with rate card	
Display score range	
Snapshots	272 (230 scheduled_refr
Weights rows	
Active social connections	
Analytics rows	connecte
Deals earned / detected	
Athletes with rate floors set	
Scored inbox threads	

ATHLETE	SCORE	PROFILE
Mia Bhuta	672 Established	full
Lara Somfai	608 Established	socialAndRese
Matt Rose	563 Established	socialAndRese
Anthony Batson Jr.	551 Established	socialAndRese
Aditya Sharma	512 Rising	stale (Jul 23 card "v3")
Chisom Okpara	506 Rising	researchOnly
Paytest Athlete	359 Local	researchOnly
Lilia Bhuta	359 Local	socialAndRese
Aaron C.-S. Nguyen	296 Local	researchOnly
demo	277 Local	no rate card

Read this as: the audience capital saturates at 100 for anyone above roughly 10K followers because of the log curve, athletic dominates for everyone else, and commercial is zero for all but two rows. Whatever the new methodology is, ten athletes is a small enough set to hand-check every before/after.

H · Things the audit found that the transition must not trip over

1. SILENT **Rates default missing capitals to 50 and unknown tier words to the lowest gate.** A renamed capital or a new tier vocabulary changes every rate with no error. rateService.ts scoreMod, prestigeTier, autographTier
2. CRASH **reloadWeights throws if the config lacks `rateEstimation.brackets`** (it sorts them unconditionally), even though nothing reads them since the v7 rate model was retired. A clean new weights row without that dead block will fail every scoring at startup. weightsService.ts reloadWeights
3. SILENT **Mila and the voice worker drop the score card if any of the four capital keys is missing.** The client rings render 0 instead. mila/widgets.ts, voice/scoreRoute.ts, mila-voice widgetBridge.ts
4. HISTORY **The whole ScoreResult is stored on every snapshot and the voice score-history route reads `displayScore` / `tier` from those rows.** A new result shape must either keep those two keys or the history reader must be versioned. 272 rows today.
5. HISTORY **The engine is path-dependent.** Five audience signals need ≥3 prior snapshots; spike and delta terms diff against the previous row. A new algorithm that is not history-dependent will make old and new scores incomparable for the first run and every "unchanged" comparison in refreshScores will fire once.
6. CONTRADICTION **Mila's system prompt states "Deals do NOT affect the M.I.A. score"**, while Commercial capital counts accepted deals and the cold-start profile flips on them (Mia +7 from gorjana). Decide which is true in the new methodology and fix the prompt to match. mila/prompt.ts line 86
7. BAKED **Deal verdicts bake the athlete's rate into prose stored on the thread.** 268 scored threads say things like "cannot compare to her $1,000 per-post floor". A rate change needs a gated re-score (`server/scripts/_rescoreImpacted.ts` is the existing pattern; it calls Kimi per thread).
8. CACHE **Nomi Cards are cached and shares are frozen.** 9 cached kits across 3 athletes and 2 frozen shares will show the old score until rebuilt (the onboarding run deletes only the generic brand_id 0 kit). Brand-tailored kits are never invalidated by a score change.
9. DEPLOY **Weights swap pattern is one-active-row.** The partial unique index means a new version must be inserted inactive and swapped in one transaction (0038 and 0042 are the templates). Dry-run every migration with BEGIN … ROLLBACK against prod and watch the deploy go green; deploy.yml re-creates CHECK lists by hand and they must mirror schema.ts.

10. STALE **Three fixtures will drift:** `__fixtures__/scoringWeights.v7.json` (the profile-switch test), the inline weights in `server/scripts/testScoring.ts`, and the `mediaKitService.golden` snapshot pinning the "Established" tier text.
11. HYGIENE **Legacy columns still have fallback readers.** `pav` and `deal_type_rates` are projections of the v3 card, but the client, chat context, deck rates and voice score still read them as fallbacks. Aditya's row still carries a July card with version "v3", the demo row has none.
12. HYGIENE **Sport and conference words live in the engine** (NATIONAL_KW, CONFERENCE_KW, POWER_CONFERENCES in two files). The no-hardcoding rule says these belong in data. The new methodology is the moment to move them.
13. OPS **The 04:30 refresh has no advisory lock and no Cloud Scheduler job**; it is an in-process cron on every instance. Fine today because of the unchanged-skip, worth locking before a heavier engine runs nightly.
14. OPS **Direct database access from this machine is gone.** The public IP changed (38.248.93.149 is not in the authorized networks); the bundled `cloud-sql-proxy.exe` on port 5440 works.

I · Transition approach with the least downstream risk

The safest shape is the one the codebase already has: a new pure engine beside the old one, selected by the active weights row, shadow-run through the existing dry-run endpoint, then cut over by swapping one row. The decision that sets the size of the job is whether the external contract survives.

1 **Fix the contract before the algorithm.** Decide, in writing, for each of: display range (200–850?), tier vocabulary (five words?), capital set (four keys, 0–100?), and whether deals feed the score. Keeping all four means the app, Mila, voice, deck and deal pipeline need no change. Any change is a named migration with its own reader list from section F.

2 **Add the engine as a second pure function** (`scoringServiceV8.ts` or an `engine` field on the weights config), returning the same `ScoreResult` type, with an explicit adapter if the internal shape differs. Keep `computeScore`'s signature so the three writers and `refreshScores.unchanged` stay untouched. Strip the dead `rateEstimation` requirement from `reloadWeights` first.

3 **Shadow-run before any write.** Insert the new weights row inactive; extend the dry-run script pattern (`scripts/_dryRunV72.ts`) to compute old and new for all 10 athletes plus the 5 golden athletes, printing score, tier, capitals and the full rate card side by side. Store the grid on the PR the way #363 did. Run `evaluateRates.ts` against the new capitals so the rate calibration is re-checked in the same pass.

4 **Re-calibrate rates deliberately, not by accident.** If the capital distribution moves (today audience saturates at 100), the scoreMod blends in `rateModelConfig.SCORE_MOD` and the tier gates need a matching bump of `RATE_MODEL_VERSION` . Either pin rates by freezing the old capitals into the rate input during the transition, or version both together.

5 **Cut over with one transaction** (the 0038/0042 migration shape), then force a full recompute: `POST /api/internal/score-refresh-cron` writes only on change, so use `refreshAthleteScore(id, { force: true })` per athlete or a one-off script. Watch the deploy to green.

6 **Re-derive what was baked.** Re-score the 268 deal threads (gated, reviewed dry run first), rebuild cached kits for all brand ids, leave frozen shares frozen by design, and refresh the three fixtures. Update the Mila prompt line about deals.

7 **Retire the fallbacks in a separate PR.** Once every current row carries the new card, drop the readers of `pav` and `deal_type_rates` and the momentum block if the new engine does not produce it.

What I need from the new methodology to size the work

- Output shape: score range, tier names, capital names and scale, anything new the UI should show.
- Inputs: anything not already on the snapshot (new APIs mean a new writer path and a new snapshot column, not just a new engine).
- History: whether it needs prior snapshots, and how many.
- Deals: whether signed deals count, and at what moment.
- Whether the rate model keeps reading the score, and which capitals it should read.
- Cadence: nightly is free today; anything heavier needs the lock in H·12.

Files that matter

LAYER	FILES

Engine and weights	server/services/scoringService.ts · weightsService.ts · scripts/seedWeightsV7.ts · migrations/0038, 0042
Rates	server/services/rateService.ts · rateModelConfig.ts · deckRates.ts · docs/rate-model-design.md · rate-model-calibration.md
Writers	server/skills/researchAthlete.ts · fetchSocialData.ts · refreshScores.ts · server/services/athletesCurrentWriter.ts · snapshotAdapter.ts · snapshotDeals.ts
Creation	server/routes/auth.ts · waitlist.ts · admin.ts · oauthCallback.ts · server/workflow/definitions/{onboarding,firstLook,researchRefresh}.ts
Tests and tooling	server/skills/emptyDealsChangesProfile.test.ts · dbSnapshotToScoring.characterization.test.ts · athletesCurrentWriter.test.ts · scripts/_dryRunV72.ts · server/scripts/{evaluateRates,_rescoreImpacted,backfillRateCards,testScoring}.ts

